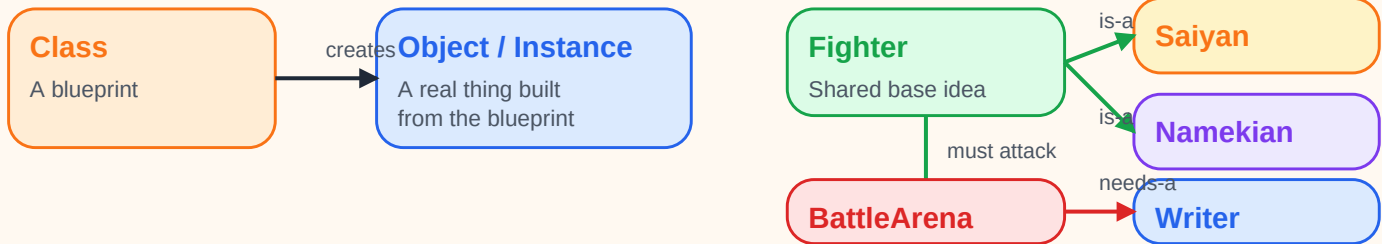


# Dragon Ball OOP Study Guide for Absolute Beginners

One-page quick reference first, then concept-by-concept notes with simple visuals and interview-friendly explanations.



## Quick Reference

### Class

A blueprint or mold used to make things.  
**In code:** `Fighter`, `Saiyan`, `Technique`

### Object

A real thing created from a class.  
**In code:** `goku`, `piccolo`, `arena`

### Instance

Another word for object.  
**In code:** `goku` is an instance of `Saiyan`

### Method

An action an object can perform.  
**In code:** `Attack()`, `TakeDamage()`

### Field

Private internal data stored inside a class.  
**In code:** `energy`

### Property

A safe door for reading or changing data.  
**In code:** `Name`, `PowerLevel`

### Encapsulation

Hide and protect internal state.  
**In code:** `energy` is `private`

### Abstraction

Show the big idea, hide the low-level details.  
**In code:** `abstract Fighter`

### Interface

A contract that says what must exist.  
**In code:** `IMessageWriter`

### Inheritance

A child class reuses a parent class.  
**In code:** `Saiyan : Fighter`

### Polymorphism

Same base type, different real behavior.  
**In code:** `Fighter goku = new Saiyan(...)`

### Composition

One object contains another object.  
**In code:** `Saiyan has Technique`

### Dependency

A class needs another piece to do its job.  
**In code:** `BattleArena` needs `writer`

### Dependency Injection

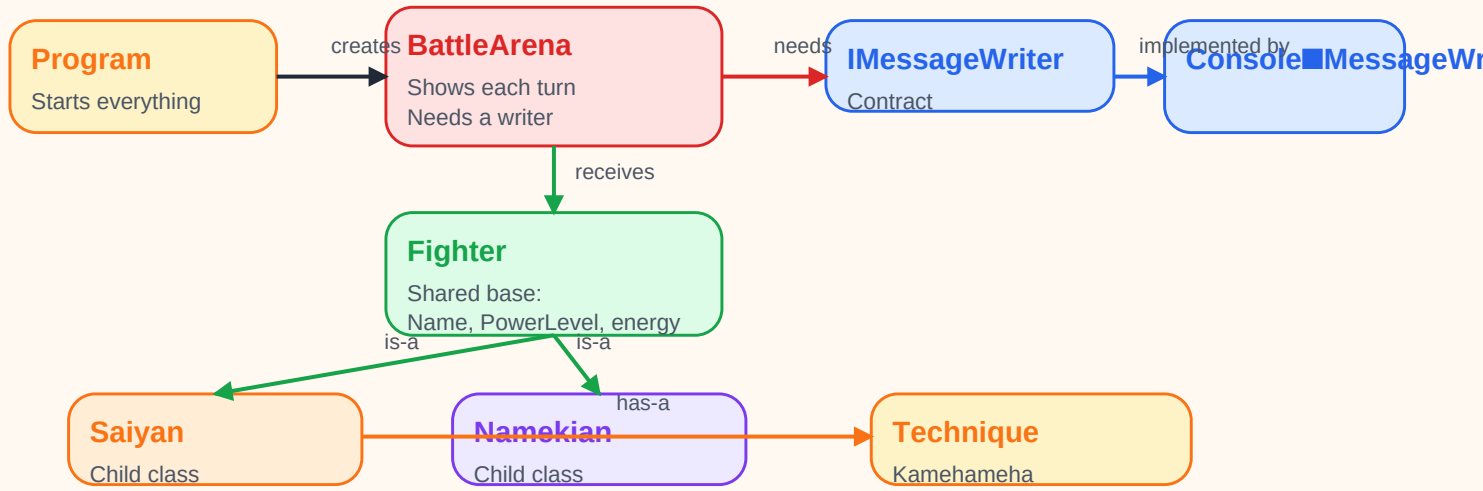
That needed piece is handed in from outside.  
**In code:** `new BattleArena(writer)`

### Tiny memory trick

Inheritance means **is-a**. Composition means **has-a**. Dependency means **needs-a**. Dependency injection means **someone else hands that needed piece in**.

# Project Map

See the whole mini-project before studying each concept one by one.



## Story of the program

- The program starts in **Main**.
- It creates a writer so text can be shown on the screen.
- It creates two fighters: one **Saiyan** and one **Namekian**.
- It creates a **BattleArena** and gives it the writer.
- The arena asks each fighter to introduce itself and attack.
- Because of polymorphism, each fighter answers in its own way.

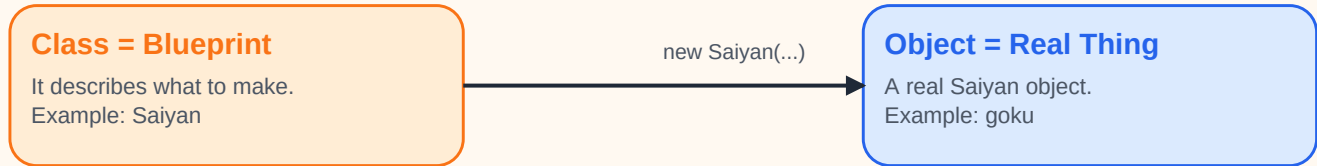
### Kid-friendly picture

Think of this project like a toy battle set. **Fighter** is the general toy idea. **Saiyan** and **Namekian** are two real toys. **BattleArena** is the stage. **IMessageWriter** is the rule that says how speech bubbles must be shown.

# Building Blocks

These are the first ideas to master: class, object, instance, method, field, and property.

## Class, Object, and Instance



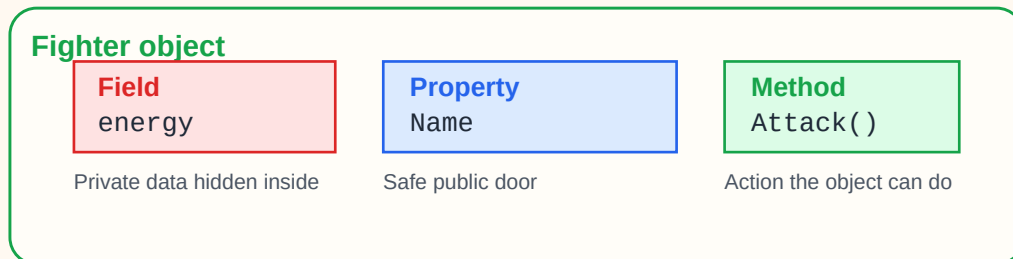
**Class:** a blueprint. It describes what we can create. In this project, **Fighter**, **Saiyan**, and **Technique** are classes.

**Object:** a real thing created from a class. When we write `new Saiyan("Goku", 9000)`, we create an object.

**Instance:** another word for object. Saying '**goku** is an instance of **Saiyan**' means the same thing as saying '**goku** is a Saiyan object.'

```
Fighter goku = new Saiyan("Goku", 9000);
```

## Method, Field, and Property



**Method:** an action. `Attack()` and `TakeDamage()` are methods.

**Field:** private internal data stored inside a class. `energy` is the field that stores a fighter's energy.

**Property:** a safe door for data. `Name` and `PowerLevel` are properties.

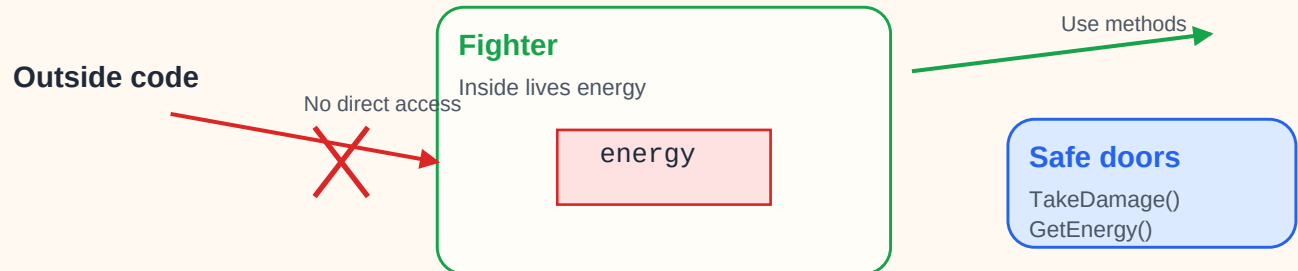
### Why field and property are different

A field is raw storage inside the object. A property is a controlled way to expose that data. In interviews, this distinction matters in C#.

# Encapsulation and Abstraction

These ideas help us design code that is safer and easier to understand.

## Encapsulation

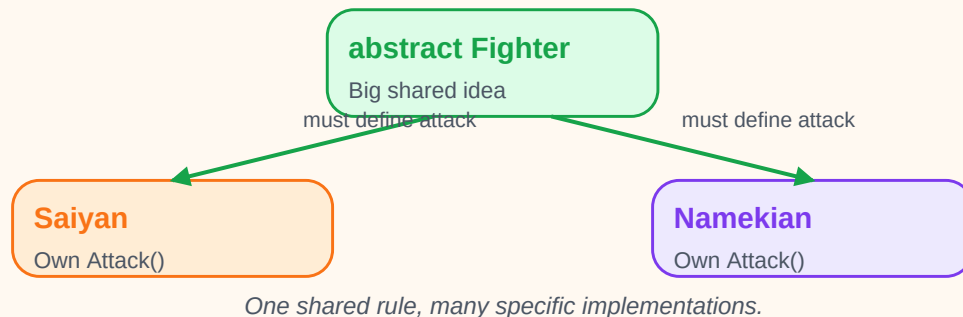


Encapsulation means the object protects its inside state. In this project, outside code cannot directly change energy because the field is private.

Instead, outside code must use safe doors such as `TakeDamage()` and `GetEnergy()`.

This protects the object from invalid states and keeps the rules in one place.

## Abstraction



Abstraction means we focus on the big idea first and leave details for later. `Fighter` says every fighter has shared data and every fighter must attack.

But `Fighter` does not decide the exact attack. Each child class decides that. That is why `Attack()` is abstract.

### Plain-English memory trick

Encapsulation says: **protect the inside**. Abstraction says: **describe the important idea first**.

# Interface, Inheritance, and Polymorphism

These three ideas are central to most OOP interviews.

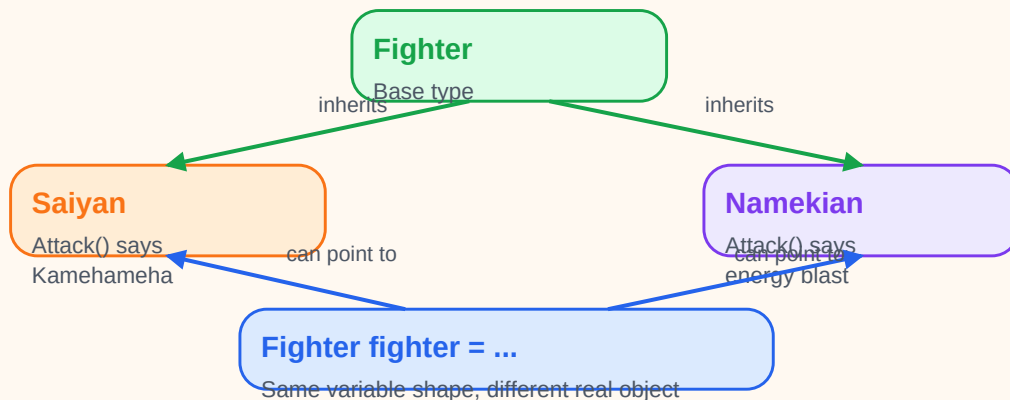
## Interface



An interface is a contract. `IMessageWriter` says a class must know how to write a message, but it does not say how.

`ConsoleMessageWriter` implements that contract by printing to the terminal.

## Inheritance and Polymorphism



Inheritance means one class builds on another. `Saiyan : Fighter` and `Namekian : Fighter` mean both child classes reuse the base class.

Polymorphism means one base type can point to different real objects. A variable typed as `Fighter` can hold a `Saiyan` or a `Namekian`.

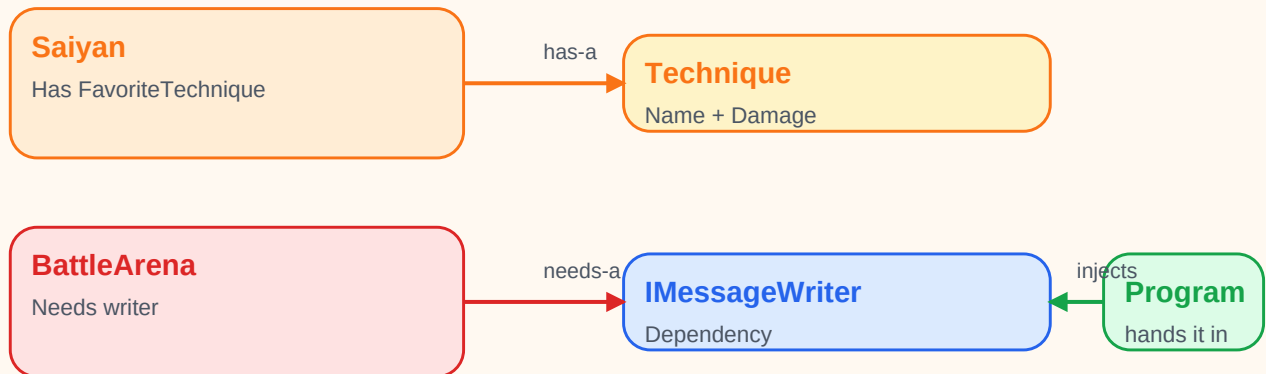
```
Fighter goku = new Saiyan("Goku", 9000);  
Fighter piccolo = new Namekian("Piccolo", 7000);
```

When the program calls `fighter.Attack()`, the answer depends on the real object behind the variable. That is polymorphism in action.

# Composition, Dependency, and Dependency Injection

These ideas explain how objects work together.

## Composition and Dependency



Composition means one object contains another object as part of itself. In this project, a Saiyan has a Technique.

Dependency means one class needs another piece to do its job. BattleArena needs something that can write messages.

## Dependency Injection

Dependency injection means the needed piece is handed in from outside instead of being created inside the class.

```
IMessageWriter writer = new ConsoleMessageWriter();
BattleArena arena = new BattleArena(writer);
```

This is called constructor injection because the dependency is passed into the constructor.

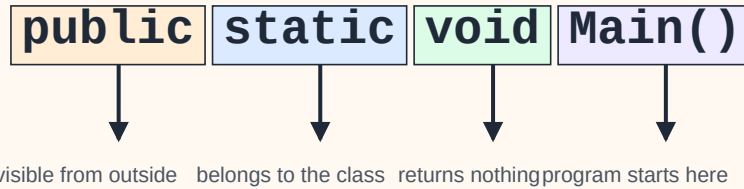
### Why this matters in interviews

Dependency injection makes code more flexible, easier to swap, and easier to test. The class depends on the **contract**, not on one hardcoded concrete implementation.

# C# Keywords Cheat Sheet

These small words carry a lot of meaning.

## Reading the signature `public static void Main()`



**public** means code from outside the class can access it. **static** means it belongs to the class itself, not to one specific object. **void** means the method returns nothing. **Main** is the special entry point where the program starts.

| Keyword   | Simple meaning                                                                         | Example in the project                  |
|-----------|----------------------------------------------------------------------------------------|-----------------------------------------|
| public    | Visible from anywhere.                                                                 | public class Fighter                    |
| private   | Visible only inside the same class.                                                    | private int energy;                     |
| protected | Visible in the class and child classes.                                                | protected Fighter(...)                  |
| abstract  | Incomplete by design; must be finished by child classes or cannot be directly created. | abstract class Fighter                  |
| virtual   | Can be replaced by a child class.                                                      | virtual string Introduce()              |
| override  | Replaces inherited behavior.                                                           | override string Attack()                |
| new       | Creates an object.                                                                     | new Saiyan("Goku", 9000)                |
| this      | The current object.                                                                    | this.writer = writer;                   |
| base      | The parent class part of the current object.                                           | base(name, powerLevel)                  |
| get / set | Read and change rules for properties.                                                  | Name { get; private set; }              |
| readonly  | Can be assigned once, usually in the constructor.                                      | private readonly IMessageWriter writer; |



# Reading the Code as a Story

If you can explain the program in order, you will sound much stronger in an interview.

## 1. Main starts

The program begins in Main.

## 2. A writer is created

ConsoleMessageWriter will print messages.

## 3. Two fighter objects are created

goku is a Saiyan. piccolo is a Namekian.

## 4. Piccolo takes damage

The method updates internal energy safely.

## 5. BattleArena is created

The writer is injected into it.

## 6. The arena shows each turn

It calls Introduce() and Attack().

## 7. Polymorphism appears

The same method call shape produces different outputs for Goku and Piccolo.

## Final memory anchor

To explain this project well, say it this way: **We model fighters with a shared abstract base class, specialize them through inheritance, swap behavior through polymorphism, protect state through encapsulation, and wire collaborating objects together through interfaces and dependency injection.**